

Декомпозиция вызовов функций во время суперкомпиляции путём построения выходных форматов

Коновалов А.В.

Метасеминар-2026, 1 июля 2026 г.

Принцип гармошки

Время доклада ограничено, а материала на слайдах много, в частности, длинные построения графов суперкомпиляции.

Поэтому доклад будет читаться так: длинные выкладки будут быстро пролистываться, желающие потом в офлайне могут потом медленно и вдумчиво изучить слайды и во всех выкладках разобраться.

Слайды будут доступны на сайте

<http://refal.botik.ru/events/events.htm>

Исходные тексты слайдов (а также обновлённую версию с исправлением опечаток) можно будет найти в репозитории:

<https://gitflic.ru/project/mazdaywik/meta2026-formats>

Принцип баяна 😊

Возможно, кто-то из слушателей заметил, что пять лет назад докладчик уже делал доклад на эту же тему:

<http://refal.botik.ru/events/IPS-RAN-MGTU-seminar-08-06-2021.pdf>

Действительно, такой доклад делался. Однако, есть и отличия:

- ▶ Предыдущий доклад излагался в терминах Рефала. В Рефале функции принимают ровно один аргумент и возвращают ровно одно возвращаемое значение. Форматы функций — приём программирования, позволяющий описывать многоместные и много-ко-местные функции. Из доклада могло создаться впечатление, что описанное расширение суперкомпиляции применимо только для Рефала, но это не так. В сегодняшнем докладе для наглядности будут использоваться обозначения Standard ML'я.
- ▶ Формулировки в докладе уточнены, приведены более наглядные примеры.
- ▶ Рассмотрены альтернативные форматы, которых не было в предыдущем докладе.

Суперкомпиляция, композиция и декомпозиция функций

Композиция функций при суперкомпиляции (1)

Известно, что суперкомпиляция неплохо справляется с вычислением композиции функций. Например, для композиции функций $f(g(\dots))$ исходной программы в остаточной программе может быть построена одна функция $fg(\dots)$.

При этом промежуточные структуры данных, порождённые вызовом $g(\dots)$ и потреблённые вызовом $f(\dots)$, в остаточной программе даже могут не строиться.

Разновидность суперкомпиляции, нацеленная на оптимизацию подобных композиций, называется *дефорестацией* (Wadler, 1990).

На вычислении подобной композиции основаны некоторые способы верификации программы при помощи суперкомпиляции.

Композиция функций при суперкомпиляции (2)

Есть более хитрые стратегии суперкомпиляции, позволяющие осуществлять *таплинг* — слияние нескольких параллельных вызовов в конфигурации, разделяющее общие переменные (Pettorossi, 1984, Chin, 1993, Secher, 2001).

Например, если в конфигурации есть пара вызовов $f(xs) \dots g(xs)$?, в остаточной программе для них построится один вызов $fg(xs)$, вычисляющий пару значений:

$$fg(xs) \equiv \langle f(xs), g(xs) \rangle$$

Подобное преобразование позволяет в некоторых случаях снижать порядок алгоритма.

Декомпозиция функций при суперкомпиляции? (1)

В классической статье (Burstall, Darlington, 1977) описывается набор правил для эквивалентной трансформации программ.

Правила, описываемые в статье обратимые. Если программу P_2 можно получить из P_1 путём эквивалентных трансформаций, то можно сделать и обратное преобразование из P_2 в P_1 .

Для того, чтобы преобразовывать программы автоматически, нужна *стратегия* применения этих правил.

Суперкомпиляция является одной из таких стратегий.

Декомпозиция функций при суперкомпиляции? (2)

Если исходная программа P_1 содержала композицию $f(g(...))$, а полученная из неё эквивалентными преобразованиями, P_2 — один вызов $fg(...)$, то применив преобразования в обратном порядке, вызов $fg(...)$ можно разложить на композицию $f(g(...))$.

Преобразование вызова $f(...)$ в эквивалентную композицию $f'(f''(...))$ мы будем называть *декомпозицией*.

Можно ли научить суперкомпиляцию делать декомпозицию?

Простые форматы функций

Определение простого формата (1)

Обозначим $RANGE(f(...))$ — область допустимых значений функции (конфигурации) $f(...)$.

Простым выходным форматом $out f$ функции $f(...)$ назовём выражение, составленное из переменных и конструкторов, которое аппроксимирует область допустимых значений этой функции, т.е. существует подстановка σ , такая что

$$\forall e \in RANGE(f(...)). \exists \sigma. e = out f / \sigma$$

Примечание. По аналогии с выходным форматом можно определить и входной, но (а) вне Рефала он не нужен (отдельные параметры можно передавать отдельными параметрами) и (б) в контексте суперкомпиляции входным форматом является само выражение базисной конфигурации.

Определение простого формата (2)

В дальнейшем понятие формата мы усложним.

Количество переменных в $outf$ мы будем называть коместностью (коарностью) $outf$, сами переменные — выходными параметрами.

Обозначим $VARs(outf)$ — функция, отображающая выходной формат на кортеж переменных в нём.

Определение простого формата (3)

Если нам известен формат функции $f(\dots)$, то мы можем определить две вспомогательные функции — будем называть их *форматными*:

$$\begin{aligned} \mathit{fmt}(\mathit{VAR}S(\mathit{outf})) &: \mathit{outf} \\ \mathit{unfmt}(\mathit{outf}) &: \mathit{VAR}S(\mathit{outf}) \end{aligned}$$

Функция $\mathit{unfmt}(\mathit{outf})$ принимает формат и извлекает из него кортеж выходных параметров. Мы будем называть её *функцией расформатирования*.

```
fun unfmt outf = VARs(outf)
```

Функция $\mathit{fmt}(\mathit{VAR}S(\mathit{outf}))$ принимает кортеж выходных параметров и строит значение выходного формата функции. Назовём её *функцией форматирования*.

```
fun fmt VARs(outf) = outf
```

Определение простого формата (4)

Очевидно, верно тождество

$$f(\dots) \equiv \textit{fmt}(\textit{unfmt}(f(\dots)))$$

Для каждой конфигурации с нетривиальным форматом суперкомпилятор будет порождать свою пару форматных функций: $\textit{fmt}_1$, $\textit{unfmt}_1$, $\textit{fmt}_2$, $\textit{unfmt}_2$ и т.д. В большинстве примеров сегодняшнего доклада конфигурация с нетривиальным форматом будет только одна, поэтому мы будем часто обходиться без индексов.

Пример: суперкомпиляция с форматами и без (1)

Рассмотрим программу, которая приписывает в конец списка три числа:

```
fun entry xs = xs ++ [1, 1, 2]
```

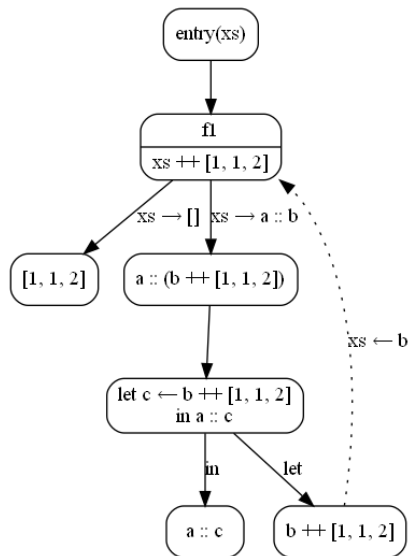
```
fun [] ++ ys = ys  
  | (x :: xs) ++ ys = x :: (xs ++ ys)
```

Условимся, что в примерах кода функция `entry` будет входной точкой программы.

Обычная суперкомпиляция, без форматов, даст скучный результат.

Пример: суперкомпиляция с форматами и без (2)

Граф суперкомпиляции для скучного примера:



Пример: суперкомпиляция с форматами и без (3)

Остаточная программа для скучного примера:

```
fun entry xs = f1(xs)  
  
fun f1 [] = [1, 1, 2]  
  | f1 (a :: b) = a :: f1(b)
```

Ничего интересного ожидаемо не получили — в точке выхода из рекурсии вместо параметра `ys` было подставлено фактическое его значение.

Пример: суперкомпиляция с форматами и без (4)

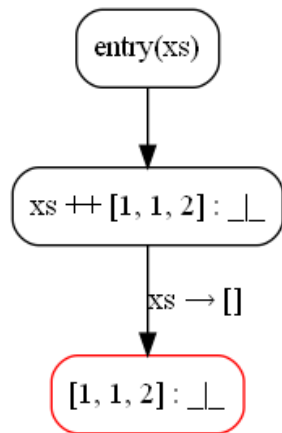
Несложно догадаться, что для конфигурации $xs \leftrightarrow [1, 1, 2]$ выходным форматом будет $a :: b :: c :: d$, ведь результатом будет список длины как минимум три. Здесь нам интересно, как суперкомпилятор это может вывести.

Форматы можно строить офлайн — по уже построенной остаточной программе, и онлайн — в процессе суперкомпиляции. В обоих случаях функциям сначала приписывается формат дна $\perp$, означающий, что функция никогда не завершается, потом в цикле эта гипотеза ослабляется до неподвижной точки.

Суперкомпилятор SCP4 строит форматы офлайн. Мы будем рассматривать построение онлайн.

При обнаружении базовой конфигурации (конфигурации, куда ведёт обратная стрелка), которой ещё не приписан формат, суперкомпилятор будет приписывать формат дна $\perp$ — делать самую сильную форматную гипотезу. Затем, при опровержении гипотезы, он её будет ослаблять, выполняя обобщение.

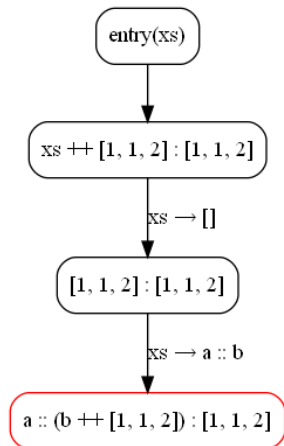
Пример: суперкомпиляция с форматами и без (5)



Форматная гипотеза
нарушилась: $[1, 1, 2]$ — вовсе
не дно, из рекурсии есть выход.
Ослабляем форматную гипотезу,
делаем обобщение:

$$[1, 1, 2] \sqcap \perp = [1, 1, 2]$$

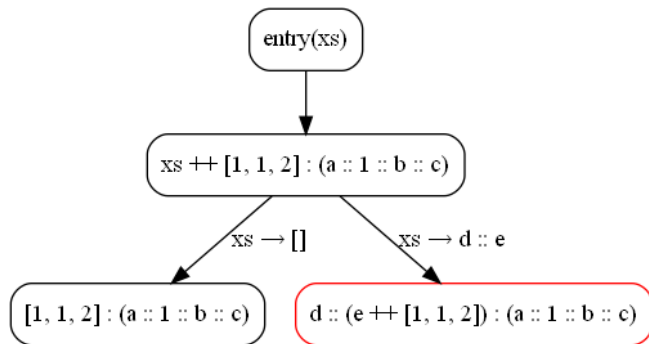
Пример: суперкомпиляция с форматами и без (6)



Теперь здесь форматная гипотеза нарушилась: формат конфигурации $a :: (b ++ [1, 1, 2])$ будет $[_ , 1, 1, 2]$. Действительно, формат $b ++ [1, 1, 2]$ согласно форматной гипотезе есть $[1, 1, 2]$ и мы ему приписываем в начало произвольный элемент. Ослабляем форматную гипотезу, снова делая обобщение:

$$[_ , 1, 1, 2] \sqcap [1, 1, 2] = a :: 1 :: b :: c$$

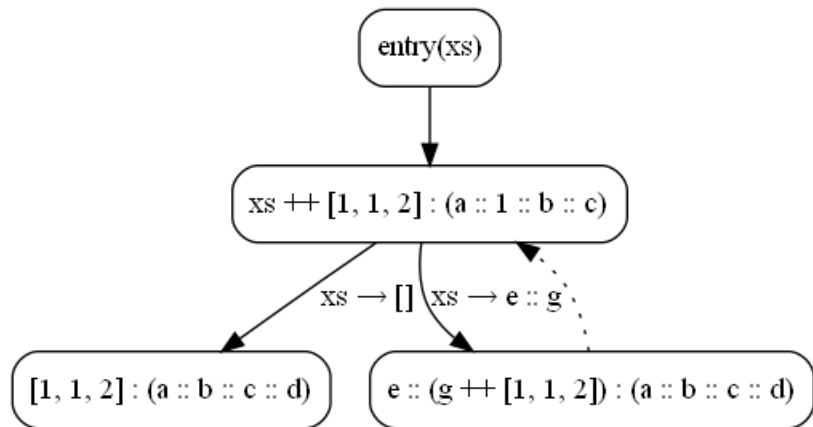
Пример: суперкомпиляция с форматами и без (7)



Здесь снова нарушилась форматная гипотеза — формат $d :: (e ++ [1, 1, 2]) : _ :: a :: 1 :: b :: c$ не соответствует ожидаемому $a :: 1 :: b :: c$, обобщаем:

$$\begin{aligned} (_ :: a :: 1 :: b :: c) \sqcap (a :: 1 :: b :: c) &= \\ &= (a :: b :: c :: d) \end{aligned}$$

Пример: суперкомпиляция с форматами и без (8)



Теперь всё верно.

Пример: суперкомпиляция с форматами и без (9)

Но сам по себе вывод форматов бесполезен, нам нужно преобразовывать программу с их использованием.

Предполагаемый суперкомпилятор сразу при построении форматной гипотезы будет строить функции форматирования и расформатирования и заменять конфигурацию, используя тождество:

$$f(\dots) \equiv \mathit{fmt}(\mathit{unfmt}(f(\dots)))$$

Конфигурация будет размыкаться в `let`-узел

$$\mathbf{let} \ (a, b, \dots) = \mathit{unfmt}(f(\dots)) \ \mathbf{in} \ \mathit{fmt}((a, b, \dots))$$

Пример: суперкомпиляция с форматами и без (10)

Конфигурация $unfmt(f(...))$ даст в остаточной программе функцию, в выходном формате которой отсутствуют избыточные конструкторы.

Конфигурация $fmt(...)$ навесит эти конструкторы на результат.

В подграфе для конфигурации $unfmt(f(...))$ все вхождения конфигурации $f(...)$ будут заменяться согласно тождеству.

Если функция где-то оказывается $unfmt(...)$ неприменима, выполняется обобщение формата.

Пример: суперкомпиляция с форматами и без (11)

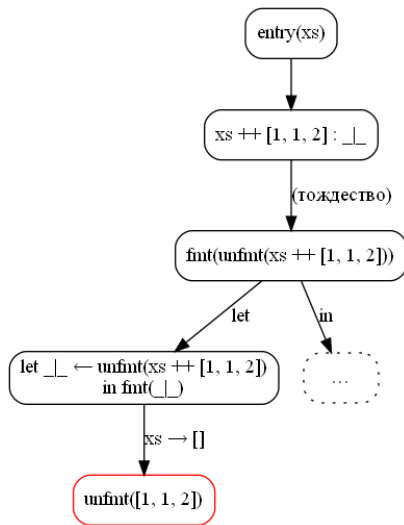
Как только обнаруживается базисная конфигурация, ей приписывается формат $\perp$ и строятся форматные функции:

fun unfmt $\perp = \perp$

fun fmt $\perp = \perp$

Функция unfmt, как видим, оказывается неприменима в красной конфигурации, обобщаем, как рассматривали ранее:

$$[1, 1, 2] \sqcap \perp = [1, 1, 2]$$



Пример: суперкомпиляция с форматами и без (12)

Новые форматные функции для нового формата:

```
fun unfmt [1, 1, 2] = ()
```

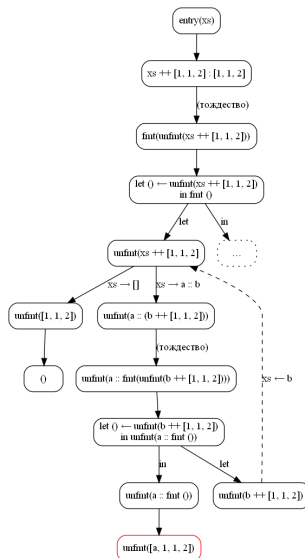
```
fun fmt () = [1, 1, 2]
```

Здесь снова наблюдаем красную конфигурацию с нарушением формата:

```
unfmt([a, 1, 1, 2])
```

Обобщаем аналогично, как рассматривали ранее, получаем формат

```
a :: 1 :: b :: c
```



Пример: суперкомпиляция с форматами и без (13)

Новые форматные функции для нового формата:

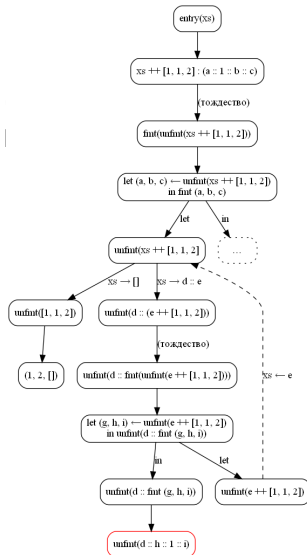
```
fun unfmt [a :: 1 :: b :: c] =  
fun fmt (a, b, c) = [a :: 1 ::
```

Здесь снова наблюдаем красную конфигурацию с нарушением формата:

```
unfmt([a, 1, 1, 2])
```

Обобщаем аналогично, как рассматривали ранее, получаем формат

```
a :: b :: c :: d
```



Пример: суперкомпиляция с форматами и без (14)

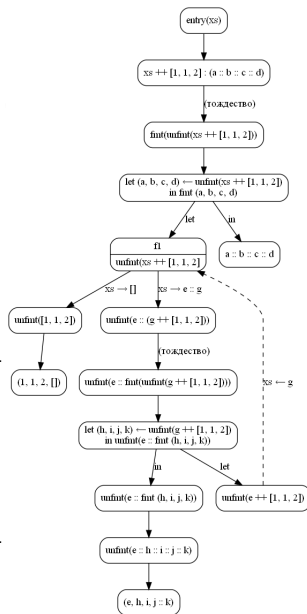
Новые форматные функции для
нового формата:

```
fun unfmt [a :: b :: c :: d] =  
fun fmt (a, b, c, d) = [a :: b
```

Теперь мы получили
законченный граф, имеем
остаточную программу:

```
fun entry xs =  
    let val (a, b, c, d) = f1  
    in a :: b :: c :: d
```

```
fun f1 [] = (1, 1, 2, [])  
    | f1 (e :: g) =  
        let val (h, i, j, k) = f1  
        in (e, h, i, j :: k)
```



Пример: суперкомпиляция с форматами и без (15)

Подытоживая, исходная программа:

```
fun entry xs = xs ++ [1, 1, 2]

fun [] ++ ys = ys
  | (x :: xs) ++ ys = x :: (xs ++ ys)
```

Остаточная программа:

```
fun entry xs =
  let val (a, b, c, d) = f1 xs
  in a :: b :: c :: d

fun f1 [] = (1, 1, 2, [])
  | f1 (e :: g) =
    let val (h, i, j, k) = f1 g
    in (e, h, i, j :: k)
```

В остаточной программе построение дополнительных трёх cons-ячеек выполняется в самом конце. Кроме того,

Пример на рекурсивный реверс (1)

Рассмотрим другой пример — рекурсивный реверс списка, в конец которого приписаны две константы:

```
fun entry xs = reverse (xs ++ [1, 2])
```

```
fun reverse [] = []  
  | reverse (x :: xs) = (reverse xs) ++ [x]
```

```
fun [] ++ ys = ys  
  | (x :: xs) ++ ys = x :: (xs ++ ys)
```

Конечно, эта функция реверса имеет квадратичную сложность, итеративно можно реверсировать за линейное время, но итеративный пример будет гораздо скучнее.

Здесь мы сразу построим окончательный граф конфигурации.

Пример на рекурсивный реверс (3)

Остаточная программа примет вид:

```
fun entry xs = 2 :: 1 :: (f1 xs)
```

```
fun f1 [] = []  
  | f1 (a :: b) =  
      let val (d, e) = f2 (f1 b, a)  
      in d :: e
```

```
fun f2 ([], a) = (a, [])  
  | f2 (g :: h, a) =  
      let val (i, j) = f2 (h, a)  
      in (g, i :: j)
```

Функция `f1` соответствует конфигурации `(reverse (xs ++ [1, 2]))`, её выходной формат был правильно вычислен и перемещён на верхний уровень (в функцию `entry`).

Функция `f2` соответствует конфигурации `(c ++ [a])`, выглядит она так страшно из-за того, что всегда возвращает непустой список, который из-за устранения формата превратился в кортеж из головы и хвоста.

Параметризованные форматы

Недостаточность простых форматов (1)

Мы только что увидели, что программа

```
fun entry xs = reverse (xs ++ [1, 2])
```

```
fun reverse [] = []  
  | reverse (x :: xs) = (reverse xs) ++ [x]
```

```
fun [] ++ ys = ys  
  | (x :: xs) ++ ys = x :: (xs ++ ys)
```

превращается в программу

```
fun entry xs = 2 :: 1 :: (f1 xs)
```

```
fun f1 [] = []  
  | f1 (a :: b) = let val (d, e) = f2 (f1 b, a) in d :: e
```

```
fun f2 ([], a) = (a, [])  
  | f2 (g :: h, a) = let val (i, j) = f2 (h, a) in (g, i :: j)
```

Недостаточность простых форматов (2)

Несложно догадаться, что программа, что программа

```
fun entry xs = reverse (xs ++ [100, 200])
```

```
fun reverse [] = []  
  | reverse (x :: xs) = (reverse xs) ++ [x]
```

```
fun [] ++ ys = ys  
  | (x :: xs) ++ ys = x :: (xs ++ ys)
```

превратится в программу

```
fun entry xs = 200 :: 100 :: (f1 xs)
```

```
fun f1 [] = []  
  | f1 (a :: b) = let val (d, e) = f2 (f1 b, a) in d :: e
```

```
fun f2 ([], a) = (a, [])  
  | f2 (g :: h, a) = let val (i, j) = f2 (h, a) in (g, i :: j)
```

Недостаточность простых форматов (3)

Если мы в программе заменим одни константы на другие, то соответствующие константы изменятся и в остаточной программе:

```
fun entry xs = reverse (xs ++ [⟨Y⟩, ⟨Z⟩])  
...
```

превращается в

```
fun entry xs = ⟨Z⟩ :: ⟨Y⟩ :: (f1 xs)  
...
```

Нам здесь интересно то, что данные константы входят в формат.

Недостаточность простых форматов (4)

Давайте попробуем заменить эти константы параметрами суперкомпиляции:

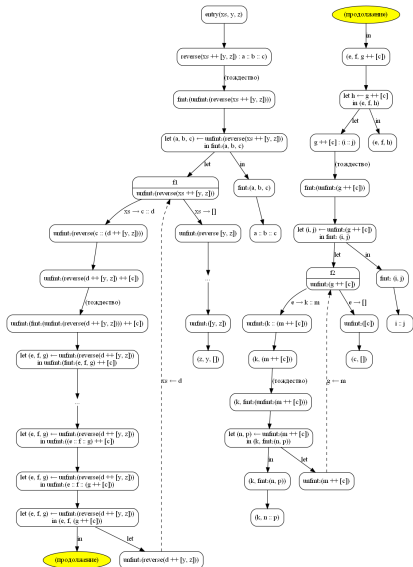
```
fun entry (xs, y, z) = reverse (xs ++ [y, z])
```

```
fun reverse [] = []  
  | reverse (x :: xs) = (reverse xs) ++ [x]
```

```
fun [] ++ ys = ys  
  | (x :: xs) ++ ys = x :: (xs ++ ys)
```

Недостаточность простых форматов (5)

Окончательный граф суперкомпиляции примет вид:



Недостаточность простых форматов (6)

Пояснения к графу. Конфигурация $\text{reverse } (xs \uparrow\uparrow [y, z])$ имеет выходной формат $a :: b :: c$, форматные функции имеют вид:

```
fun unfmt1 (a :: b :: c) = (a, b, c)
```

```
fun fmt1 (a, b, c) = a :: b :: c
```

Конфигурация $(g \uparrow\uparrow [c]) : (i :: j)$ имеет тот же выходной формат, что и в предыдущем примере и те же форматные функции:

```
fun unfmt2 (i :: j) = (i, j)
```

```
fun fmt2 (i, j) = i :: j
```

Недостаточность простых форматов (7)

Остаточная программа примет вид

```
fun entry (xs, y, z) =  
    let val (a, b, c) = f1 (xs, y, z)  
    in a :: b :: c
```

```
fun f1 ([], y, z) = (z, y, [])  
  | f1 (c :: d, y, z) =  
    let val (e, f, g) = f1 (d, y, z)  
    in let val (i, j) = f2 (g, c)  
       in (e, f, i :: j)
```

```
fun f2 ([], c) = (c, [])  
  | f2 (k :: m, c) =  
    let val (n, p) = f2 (m, c)  
    in (k, n :: p)
```

Функция f2 та же, что и в предыдущем примере, с точностью до переименования переменных.

Недостаточность простых форматов (8)

Интерес представляет функция `f1`, полученная из конфигурации

```
reverse (xs ++ [y, z]) : (a :: b :: c)
```

Посмотрим на неё ещё раз:

```
fun f1 ([], y, z) = (z, y, [])  
  | f1 (c :: d, y, z) =  
    let val (e, f, g) = f1 (d, y, z)  
    in let val (i, j) = f2 (g, c)  
      in (e, f, i :: j)
```

Видно, что параметры `y` и `z` в вычислениях не участвуют, а передаются в рекурсивные вызовы и возвращаются из них под псевдонимами `e` и `f`.

Можно это *условно (пока условно)* записать следующим форматом:

```
reverse (xs ++ [#y, #z]) : (#z :: #y :: c)
```


Параметризованные форматы (1)

Формат, содержащий параметры, помеченные «#», будем называть *параметризованным*:

`reverse (xs ++ [#y, #z]) : (#z :: #y :: c)`

Параметры с «#» будем называть *замороженными*.

Замороженные параметры могут появиться только в подграфе некоторой базисной конфигурации с форматом.

Параметризованные форматы (2)

```
reverse (xs ++ [#y, #z]) : (#z :: #y :: c)
```

Смысл замороженных параметров в том, что они в своём подграфе рассматриваются не какобычные параметры, а как некие неизвестные константы уровнем выше (как имена макросов препроцессора Си выше имён переменных в Си).

Замороженные параметры нельзя сужать при прогонке и нельзя обобщать: они могут только передаваться неизменными из конфигурации в конфигурацию и возвращаться как часть формата.

Параметризованные форматы (3)

Для форматов с замороженными параметрами особым образом строятся форматные функции:

```
reverse (xs ++ [#y, #z]) : (#z :: #y :: c)
```

```
fun unfmt (#z :: #y :: c) = c
```

```
fun fmt (c, y, z) = (z :: y :: c)
```

Функция форматирования `fmt` принимает форматные параметры как обычные, для функции расформатирования форматные параметры — константы, подлежащие забвению.

Действительно, раз в вычислениях они не участвуют, их можно удалить из формата, вернув кортеж только реально вычисленных значений. Однако, при обёртывании результата в формат они нужны.

Параметризованные форматы (4)

```
reverse (xs ++ [#y, #z]) : (#z :: #y :: c)
```

```
fun unfmt (#z :: #y :: a) = a
```

```
fun fmt (a, y, z) = (z :: y :: a)
```

Тождество будет выглядеть так:

```
reverse (xs ++ [#y, #z]) ≡  
  ≡ fmt (unfmt (reverse (xs ++ [#y, #z])), #y, #z)
```

Однако, после преобразования в let-узел на верхнем уровне аргументы функции `fmt` должны быть разморожены.

Параметризованные форматы (5)

Но откуда брать эти параметризованные форматы?

Если при суперкомпиляции при прогонке и при обобщении всегда создаются свежие имена параметров, то при обнаружении зацикливания в базисную конфигурацию параметры со старыми именами будут кандидатами в замороженные.

Если при анализе конфигурации с замороженными параметрами их приходится сужать или обобщать, то форматная гипотеза ослабляется — соответствующие параметры «размораживаются» и становятся обычными.

Другой вариант (который рассмотрим позже): превентивно замораживать новые параметры, возникающие при опасном сходстве и обобщении.

Пример на параметризованные форматы (1)

Попробуем просуперкомпилировать эту же программу с использованием замороженных параметров — параметризованных форматов:

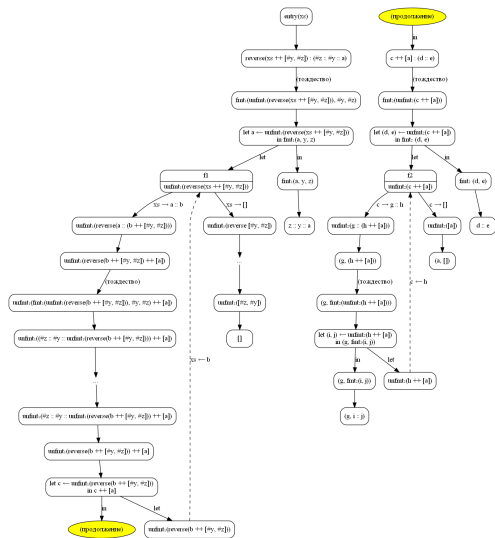
```
fun entry (xs, y, z) = reverse (xs ++ [y, z])
```

```
fun reverse [] = []  
  | reverse (x :: xs) = (reverse xs) ++ [x]
```

```
fun [] ++ ys = ys  
  | (x :: xs) ++ ys = x :: (xs ++ ys)
```

Пример на параметризованные форматы (2)

Граф суперкомпиляции для этого примера:



Пример на параметризованные форматы (3)

Форматные функции в этом примере примут вид:

```
fun unfmt1 (#z :: #y :: a) = a
```

```
fun fmt1 (a, y, z) = z :: y :: a
```

```
fun unfmt2 (d :: e) = (d, e)
```

```
fun fmt2 (d, e) = d :: e
```


Пример на параметризованные форматы (4)

Остаточная программа примет вид:

```
fun entry (xs, y, z) = z :: y :: (f1 xs)
```

```
fun f1 [] = []  
  | f1 (a :: b) = let val (d, e) = f2 (f1 b, a) in d :: e
```

```
fun f2 ([], a) = (a, [])  
  | f2 (g :: h, a) =  
    let val (i, j) = f2 (h, a)  
    in (g, i :: j)
```

Параметры программы y и z в вычислениях не участвуют, выносятся наружу прямо во входной точке.

Пример на параметризованные форматы (5)

Нетрудно убедиться, что при построении графа суперкомпиляции параметры $\#x$ и $\#y$ конфигурации `reverse` ($xs \mapsto [\#y, \#z]$) так и останутся замороженными:

- ▶ В ветви прогонки по $xs \rightarrow a :: b$ обнаружится конфигурация `reverse` ($b \mapsto [y, z]$) $\mapsto [a]$, в ней параметр xs превратился в b , тогда как параметры y и z не переименовались и остались на своих местах — кандидаты на заморозку.
- ▶ Выход из рекурсии по $xs \rightarrow []$ будет иметь вид $[\#z, \#y]$.
- ▶ В рекурсивной ветви по $xs \rightarrow a :: b$ конфигурация `reverse` ($b \mapsto [\#y, \#z]$) $\mapsto [a]$ получит формат $[\#z, \#y, _]$, который обобщится до $\#x :: \#y :: _$. Этот формат и станет окончательным.

Пример на параметризованные форматы (6)

Однако, в базисной конфигурации $c \leftrightarrow [a]$ параметр a будет разморожен. Действительно:

- ▶ При сужении $c \rightarrow g :: h$ получим $g :: (h \leftrightarrow [a])$, заикливание, параметр c переименовался в h , однако, a так и остался — кандидат на заморозку.
- ▶ При суперкомпиляции конфигурации $h \leftrightarrow [\#a]$ выходом из рекурсии по $c \rightarrow []$ будет $[\#a]$, которая образует первую форматную гипотезу.
- ▶ Во второй ветви прогонки $c \rightarrow g :: h$ получим $g :: (h \leftrightarrow [\#a])$, имеющую формат $[_ , \#a]$ будет обобщён с $[\#a]$, замороженный параметр придётся разморозить.

Активные форматы

Активные форматы (1)

Мы только что рассмотрели, что

- ▶ форматы могут быть простыми,
- ▶ форматы могут быть параметризованными.

Активные форматы (1)

Мы только что рассмотрели, что

- ▶ форматы могут быть простыми,
- ▶ форматы могут быть параметризованными.

Идём дальше:

- ▶ форматы могут быть активными.

Т.е. форматы могут содержать не только конструкторы, параметры (в т.ч. и замороженные), но и вызовы функций!

Активные форматы (2)

Откуда они могут взяться и что с ними делать?

Для начала, откуда они могут взяться. Мы говорили, что замороженные параметры нельзя сужать и обобщать. Поэтому, если мы получаем вызов функции, вычисление которой зависит от замороженного параметра, мы можем параметр разморозить и вызов прогнать.

А можем не прогонять, а просто сказать, что вот этот вызов и будет выходом из рекурсии! Вызов заморозится вместе с параметром, станет частью формата (на него можно будет смотреть как на особого рода конструктор), после расформатирования конфигурация будет возвращать незамороженные параметры формата, в т.ч. и этого вызова.

Соответственно, потом, форматируя возвращаемое значение, мы эту функцию вызовем и таким образом добьёмся титульной цели доклада — декомпозиции функций!

Активные форматы (3)

Заметим, что отложенные вызовы функций в формате появляются, когда функция не может быть вычислена из-за заморозки своего аргумента. Таким образом, любой активный формат параметризованный.

Дурацкий пример на активные форматы (1)

Сложно подобрать короткий, но красивый пример чисто на активные форматы. Поэтому пример будет дурацкий. Припишем к списку списков ещё один список и создадим список голов в обратном порядке:

```
fun entry (xss, xs) = reverseHeads (xss ++ [xs])
```

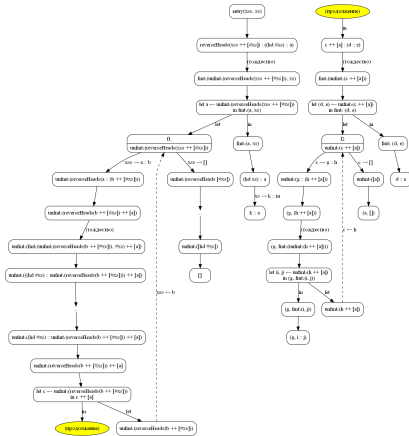
```
fun hd (x :: xs) = x
```

```
fun reverseHeads [] = []  
  | reverseHeads (xs :: xss) =  
    (reverseHeads xss) ++ [hd xs]
```

```
fun [] ++ ys = ys  
  | (x :: xs) ++ ys = x :: (xs ++ ys)
```

Позже будут примеры сложнее и красивее.

Дурацкий пример на активные форматы (2)



Конфигурация с ++ [a] будет
иметь формат d :: e, её
форматные функции:

```
fun unfmt2 (d :: e) = (d, e)
```

```
fun fmt2 (d, e) = d :: e
```

Ситуация с конфигурацией `reverseHeads (xss ++ [#hd])` интереснее. Её формат `(hd #xs) :: a`, а форматные функции, соответственно:

```
fun unfmt1 ((hd #xs) :: a) = a
```

```
fun fmt1 (a, xs) = (hd xs) :: a
```

Дурацкий пример на активные форматы (3)

Ещё разок, конфигурация `reverseHeads (xss ++ [#hd])` имеет формат

`(hd #xs) :: a`

Формат содержит невычисленный вызов функции `hd #xs`, который «застопорился» на замороженном параметре.

Форматные функции для него имеют вид:

fun `unfmt1 ((hd #xs) :: a) = a`

fun `fmt1 (a, xs) = (hd xs) :: a`

Любопытно тут то, что функция расформатирования содержит в левой части вызов функции! Такое на Standard ML написать невозможно, но в суперкомпиляции мы это разрешаем — функция расформатирования обрабатывается особым образом.

Дурацкий пример на активные форматы (4)

Остаточная программа будет похожа на остаточные программы предыдущих примеров с реверсом:

```
fun entry (xss, (k :: m)) = k :: (f1 xss)
```

```
fun f1 [] = []  
  | f1 (a :: b) =  
      let val (d, e) = f2 (f1 b, a)  
      in d :: e
```

```
fun f2 ([], a) = (a, [])  
  | f2 (g :: h, a) =  
      let val (i, j) = f2 (h, a)  
      in (g, i :: j)
```

Здесь мы видим, что второй параметр входной точки сузился прямо в ней же.

Активные форматы и моноиды

Добавим в суперкомпилятор поддержку моноида (1)

Несколько компактных красивых примеров с активными форматами можно получить, если позволить суперкомпилятору преобразовывать моноиды. Добавим в наш гипотетический суперкомпилятор поддержку моноида арифметического умножения, т.е. будем предполагать, что при преобразованиях можно пользоваться аксиомами:

$$a \times (b \times c) = (a \times b) \times c = a \times b \times c$$

$$a = a \times 1 = 1 \times a$$

Других моноидов нам не потребуется, следующим двум примерам достаточно одного умножения.

Примечание. Суперкомпиляторы с поддержкой моноида существуют в действительности.

Добавим в суперкомпилятор поддержку моноида (2)

Для наглядности, умножение, которое «понимает» суперкомпилятор, будем обозначать как « $\times$ ».

Поддержка моноида умножения означает, что

- ▶ суперкомпилятор может обобщать конфигурации вида a и $a \times b$ как $a \times c$, где $c \rightarrow 1$ и $c \rightarrow b$ для обоих исходных конфигураций;
- ▶ суперкомпилятор может пользоваться ассоциативностью умножения.

Пример: итеративный факториал (1)

Рассмотрим пример: итеративное вычисление факториала:

```
fun entry n = loop (1, n)
```

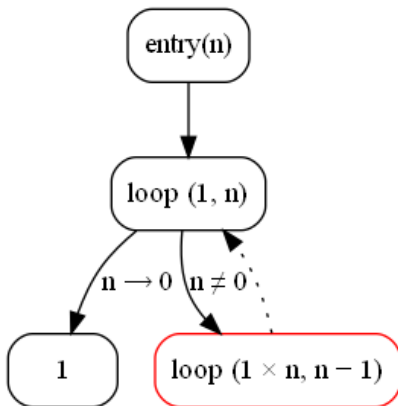
```
fun loop (acc, 0) = acc  
  | loop (acc, n) = loop (acc × n, n - 1)
```


Пример: итеративный факториал (2)

Начнём строить граф суперкомпиляции. Обнаружили опасное сходство. Тут, вопреки сказанному ранее, параметр аккумулятора обобщим и сразу заморозим.

Почему тут нужно не обобщать $1 \sqcap 1 \times n = 1 \times x$, объяснить трудно.

Гипотеза выходного формата дефолтовая — $\perp$.



Пример: итеративный факториал (3)

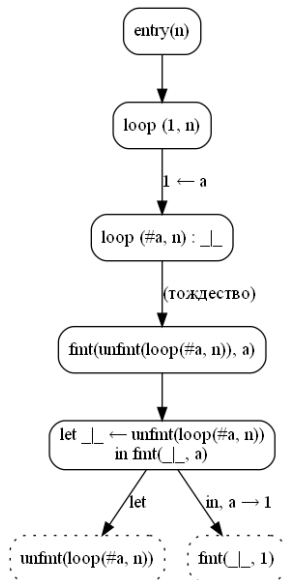
Здесь мы обобщили аккумулятор, заморозили его, применили тождество, разобрали тождество let-узлом и начали прогонять обе конфигурации отдельно.

Заметим, что в ветке `in` мы «вспомнили» исходное значение аккумулятора. Это делать не обязательно, просто в остаточной программе будут лишние вычисления, которые придётся устранять повторной суперкомпиляцией.

Форматные функции очевидны:

```
fun unfmt  $\perp$  =  $\perp$ 
```

```
fun fmt  $\perp$  =  $\perp$ 
```



Пример: итеративный факториал (4)

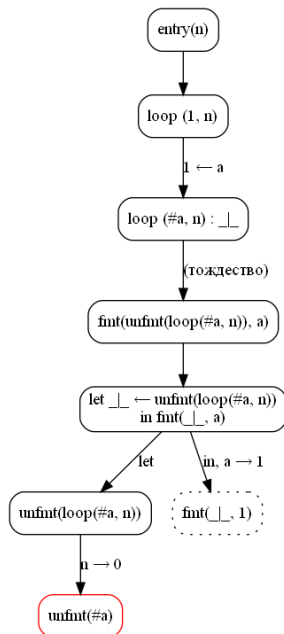
Форматные функции очевидны:

fun unfmt $\perp = \perp$

fun fmt $\perp = \perp$

В красной конфигурации имеем нарушение формата, делаем обобщение:

$$\perp \sqcap \#a = \#a$$



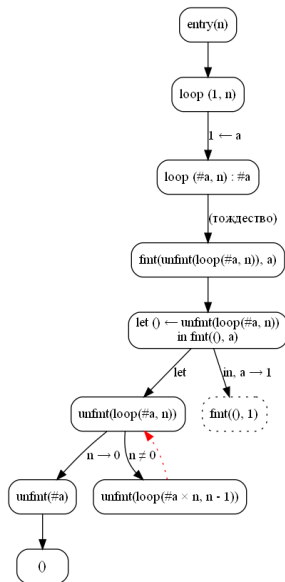
Пример: итеративный факториал (5)

Форматные функции:

fun unfmt #a = ()

fun fmt ((), a) = a

Можно было не передавать (),
мы её написали для
наглядности.



Пример: итеративный факториал (6)

Мы получили конфигурацию `unfmt(loop(#a × n, n - 1))`. Что там с ней делать дальше? Должны применить тождество.

Вспомним формат $loop(\#a', n) : \#a'$ (штрихи — см. далее). Форматная гипотеза предполагает, что функция `loop` всегда возвращает значение своего аккумулятора как есть, аккумулятор в вычислениях не участвует (очевидно, гипотеза неточная, всё впереди).

Применим тождество:

```
unfmt(loop(#a × n, n - 1), #a)
unfmt(fmt(unfmt(loop(#a × n, n - 1)), #a × n))
```

Здесь мы рассмотрели фактический аргумент `#a × n` как параметр `#a'` формата.

Пример: итеративный факториал (7)

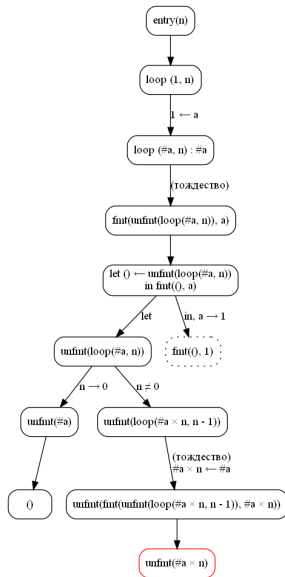
Форматные функции:

fun unfmt #a = ()

fun fmt ((), a) = a

В красной конфигурации нарушается формат, выполняем обобщение:

$$\#a \sqcap \#a \times n = \#a \times b$$



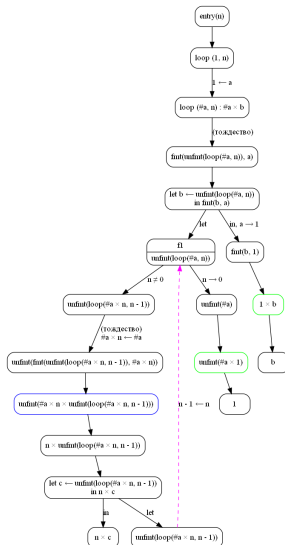
Пример: итеративный факториал (8)

Форматные функции:

fun unfmt #a × b = b

fun fmt (b, a) = a × b

Граф суперкомпиляции
построен полностью.



Пример: итеративный факториал (9)

Несколько замечаний по построенному графу.

- ▶ Формат конфигурации `loop (#a, n)` : $\#a \times b$ говорит о том, что начальное значение аккумулятора не влияет на ход вычислений, а сразу отражается на конечном результате.
- ▶ В зелёных конфигурациях мы пользуемся аксиомой умножения на единицу — в конфигурации `unfmt(#a × 1)` мы единицу добавляем, в $1 \times b$ — удаляем.
- ▶ В синей конфигурации мы пользуемся ассоциативностью моноида.
- ▶ Конфигурации, соединённые сиреневой штриховой обратной стрелкой, различаются обоими параметрами функции `loop`, но на стрелке указан только один $n - 1 \leftarrow n$. Причина в том, что второй параметр заморожен, а значит, согласно форматной гипотезе на вычисления не влияет.
- ▶ Ветка `in` у самого верхнего `let`-узла выродилась в `b`.

Пример: итеративный факториал (10)

Остаточная программа примет вид:

```
fun entry n = f1 n
```

```
fun f1 0 = 1  
  | f1 n = n × (f1 (n - 1))
```

Мы получили из итеративной классическую очевидную рекурсивную версию факториала!

Пример: произведение конкатенации (1)

Второй наглядный пример — произведение элементов конкатенации двух списков:

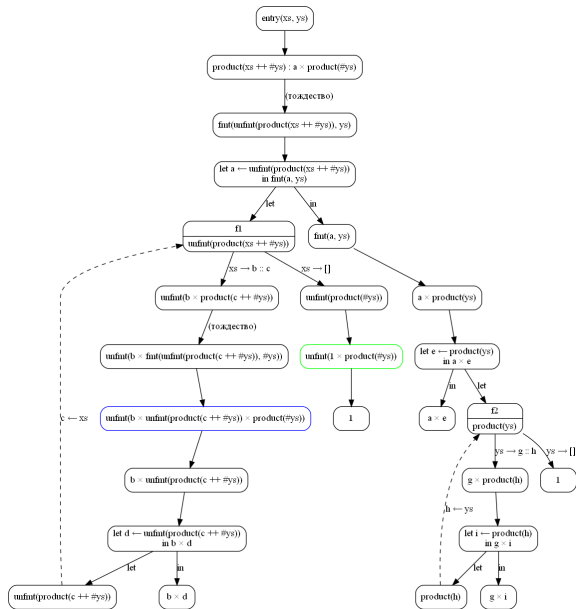
```
fun entry (xs, ys) = product (xs ++ ys)
```

```
fun product (n :: ns) = n * (product ns)  
  | product [] = 1
```

```
fun (x :: xs) ++ ys = x :: (xs ++ ys)  
  | [] ++ ys = ys
```

Пример: произведение конкатенации (2)

Сразу окончательный граф суперкомпиляции:



Пример: произведение конкатенации (3)

Замечания к графу

- ▶ В синей конфигурации мы воспользовались ассоциативностью умножения, в зелёной — свойством умножения на единицу.
- ▶ У конфигурации `product(ys)` выходной формат тривиальный, поэтому форматные функции не создавались.
- ▶ Форматные функции для конфигурации `product(xs ++ #ys) : a × product(#ys)`:

```
fun unfmt (a × product(#ys)) = a
fun fmt (a, ys) = a × product(ys)
```
- ▶ Формат конфигурации `product(xs ++ #ys)` получен за несколько итераций:
 1. `product(xs ++ #ys) : ⊥`
 2. `product(xs ++ #ys) : product(#ys)`
 3. `product(xs ++ #ys) : a × product(#ys)`

Пример: произведение конкатенации (4)

Остаточная программа примет вид:

```
fun entry (xs, ys) = (f1 xs) × (f2 ys)
```

```
fun f1 (b :: c) = b × (f1 c)  
  | f1 [] = 1
```

```
fun f2 (g :: h) = g × (f2 h)  
  | f2 [] = 1
```

Функции $f1$ и $f2$ эквивалентны исходной функции $product$.
Причём для $f2$ это ожидаемо (она получена из конфигурации $product(ys)$), то функция $f1$ родилась в процессе декомпозиции.

Содержательно можно заметить, что мы доказали:
произведение элементов конкатенации списков равно
произведению произведений элементов каждого из списков.

Большой пример — суперкомпиляция
интерпретатора

Интерпретатор (1)

Этот пример не требует моноида умножения, ему достаточно только активных форматов.

Рассмотрим интерпретатор стекового языка программирования, который описан при помощи итеративной функции, хранящей стек вызовов и стек данных в аккумуляторах.

Для упрощения выкладок, словарь глобальный. Его можно было сделать параметром интерпретатора, но это загромождает изложение, ничего принципиального не добавив.

Интерпретатор (2)

Стековые команды:

datatype Command =

DUP	-- дублирует вершину стека
DROP	-- удаляет вершину стека
CONST int	-- кладёт константу на стек
MUL	-- перемножает вершину и подвершину
SUB	-- вычитает из подвершины вершину
IF_ [Command] * [Command]	-- если на стеке 0 , на стек возврат
	-- кладётся вторая ветка, иначе первая
CALL string	-- на стек возвратов кладётся код
	-- функции с соответствующим именем

Интерпретатор (3)

Функция интерпретации принимает два параметра:

- ▶ стек данных [int],
- ▶ стек возвратов [[Command]].

Стек возвратов представлен как список списков команд — при вызове функции на него помещается тело вызванной функции, при вызове ветки IF_’а — соответствующая ветка.

Интерпретатор (4)

Функция интерпретации (начало):

```
fun loop (x :: xs, (DUP :: cs) :: css) =  
    loop (x :: x :: xs, cs :: css)  
| loop (x :: xs, (DROP :: cs) :: css) =  
    loop (xs, cs :: css)  
| loop (xs, ((CONST x) :: cs) :: css) =  
    loop (x :: xs, cs :: css)  
| loop (a :: b :: xs, (MUL :: cs) :: css) =  
    loop ((a × b) :: xs, cs :: css)  
| loop (a :: b :: xs, (SUB :: cs) :: css) =  
    loop ((b - a) :: xs, cs :: css)
```

...

Интерпретатор (5)

Функция интерпретации (окончание):

...

```
| loop (0 :: xs, ((IF_ tcs fcs) :: cs) :: css) =  
    loop (xs, fcs :: cs :: css)  
| loop (x :: xs, ((IF_ tcs fcs) :: cs) :: css) =  
    loop (xs, tsc :: cs :: css)  
| loop (xs, ((CALL name) :: cs) :: css) =  
    loop (xs, (dict name) :: cs :: css)  
| loop (xs, [] :: css) = loop (xs, css)  
| loop (xs, []) = xs
```

Интерпретатор (6)

Входная точка:

```
fun entry n = loop ([n, 7], [(CALL "fact"), SUB])
```

Входная точка вычисляет факториал для аргумента и вычитает 7.

Вычитание семи добавлено для наглядности: донья стеков будут непустыми.

Интерпретатор (7)

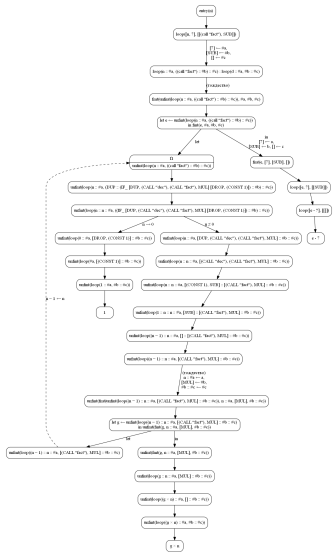
Словарь будет иметь вид

```
fun dict "fact" = [  
    DUP,  
    (IF_  
        [DUP, (CALL "dec"), (CALL "fact"), MUL]  
        [DROP, (CONST 1)]  
    )  
]  
| dict "dec" = [(CONST 1), SUB]
```

Заметим, что функция факториала описана рекурсивно — на каждом её вызове оба стека растут, рекурсия не хвостовая.

Интерпретатор (8)

Сразу построим полный граф суперкомпиляции:



Интерпретатор (9)

Замечания к графу:

- ▶ Форматные функции:

```
fun unfmt (loop(d :: #a, #b :: #c) = d
```

```
fun fmt (d, a, b, c) = loop(d :: a, b :: c)
```

- ▶ Граф построен с небольшим читеством — базисная конфигурация не глобальная. Можно было бы честно сделать базисную конфигурацию глобальной, но это немного увеличило бы граф.

Интерпретатор (10)

Остаточная программа примет вид:

```
fun entry n = (f1 n) - 7
```

```
fun f1 0 = 1  
  | f1 n = (f1 (n - 1)) × n
```

Остаточная программа представляет собой рекурсивный факториал, семантически эквивалентна интерпретируемой программе на стековом языке.

Терминологии интерпретатора в остаточной программе нет, произошла чистая компиляция.

Классическая суперкомпиляция Сёренсена для данной программы была бы бессильна: аккумуляторы обобщились бы, остаточная программа была бы эквивалентна исходному интерпретатору (разве что вызовы `dict` заинлайнились).

Альтернативные форматы

Альтернативные форматы (1)

Мы только что рассмотрели, что

- ▶ форматы могут быть простыми,
- ▶ форматы могут быть параметризованными,
- ▶ форматы могут быть активными.

Альтернативные форматы (1)

Мы только что рассмотрели, что

- ▶ форматы могут быть простыми,
- ▶ форматы могут быть параметризованными,
- ▶ форматы могут быть активными.

Идём дальше:

- ▶ форматы могут быть альтернативными.

Под альтернативным форматом будем подразумевать что у функции будет несколько разных способов вернуть результат.

Альтернативные форматы (2)

Поскольку любые два пассивных выражения можно обобщить (в худшем случае до переменной), альтернативные форматы возникают, когда либо есть активный и пассивный возвраты (выходы из рекурсии), либо два активных возврата с двумя разными вызовами застопоренных функций.

Таким образом, любой альтернативный формат будет активным форматом.

Также здесь любопытно то, что в форматных функциях будет несколько ветвей — по одной на каждую альтернативу.

Пример на альтернативные форматы (1)

Напишем функцию `first_not_zero`, которая в списке чисел находит первое ненулевое значение. Если таковое найдено, функция возвращает список длины 1 из найденного числа, иначе возвращается пустой список.

В примере мы найдём первое ненулевое число в конкатенации двух списков:

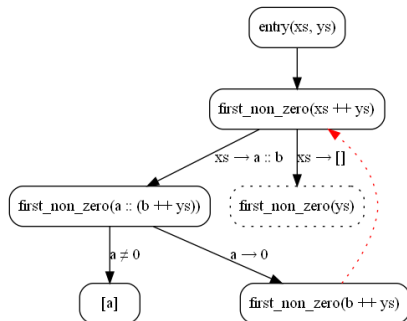
```
fun entry (xs, ys) = first_non_zero (xs ++ ys)

fun first_non_zero [] = []
  | first_non_zero (0 :: ns) = first_non_zero ns
  | first_non_zero (n :: ns) = [n]

fun (x :: xs) ++ ys = x :: (xs ++ ys)
  | [] ++ ys = ys
```

Пример на альтернативные форматы (2)

Обнаружили опасное сходство (да чего греха таить, заикливание) между конфигурациями `first_non_zero(xs ++ ys)` и `first_non_zero(b ++ ys)`. В конфигурациях не менялся параметр `ys` — кандидат на заморозку. Назначаем конфигурации формат $\perp$.



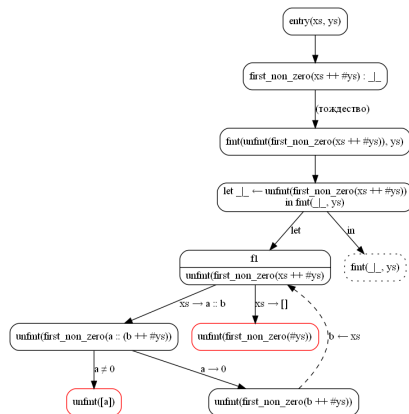
Пример на альтернативные форматы (3)

Форматные функции

fun unfmt $\perp = \perp$

fun fmt $\perp = \perp$

Здесь мы имеем сразу два выхода из рекурсии с нарушением формата (два фейла в вызовах unfmt), обе конфигурации обозначены красным цветом. Возникает вопрос: а как их обобщить?



$\perp \sqcap [a] \sqcap \text{first_non_zero}(\#ys) = ?$

Пример на альтернативные форматы (4)

$$\perp \sqcap [a] \sqcap \textit{first_non_zero}(\#ys) = ?$$

Можно обобщить до тривиального формата, но это не интересно.

Одна ветка здесь имеет простой формат, другая — активный, объединить их никак нельзя. Но раз мы их не можем слить воедино, оставим их в виде альтернативы. Объединение форматов будем обозначать вертикальной чертой:

$[a] \mid \textit{first_non_zero}(\#ys)$

Такие форматы будем называть *альтернативными*.

Пример на альтернативные форматы (5)

```
[a] | first_non_zero(#ys)
```

На первый взгляд непонятно, как написать форматные функции. Можно предположить (и это будет верно), что эти функции уже получают по два предложения и будут иметь вид:

```
fun unfmt [a] = ???  
  | unfmt first_non_zero(#ys) = ???
```

```
fun fmt ??? = [a]  
  | fmt ??? = first_non_zero(#ys)
```

Но что же на месте знаков вопросов? Раньше там были обыкновенные кортежи (кортеж из нуля элементов — `unit`, из одного — сам этот элемент). Здесь придётся ввести размеченное объединение...

Пример на альтернативные форматы (6)

Для альтернативного формата

```
[a] | first_non_zero(#ys)
```

опишем такой алгебраический тип данных:

```
datatype first_non_zero_ret = R1 int | R2
```

Первый конструктор описывает возврат элемента списка `[a]`, поэтому содержит параметр типа `int`. Второй параметров не содержит, т.к. в `first_non_zero(#ys)` только замороженные параметры.

Пример на альтернативные форматы (7)

```
[a] | first_non_zero(#ys)
```

Функции форматирования и расформатирования примут вид

```
datatype first_non_zero_ret = R1 int | R2
```

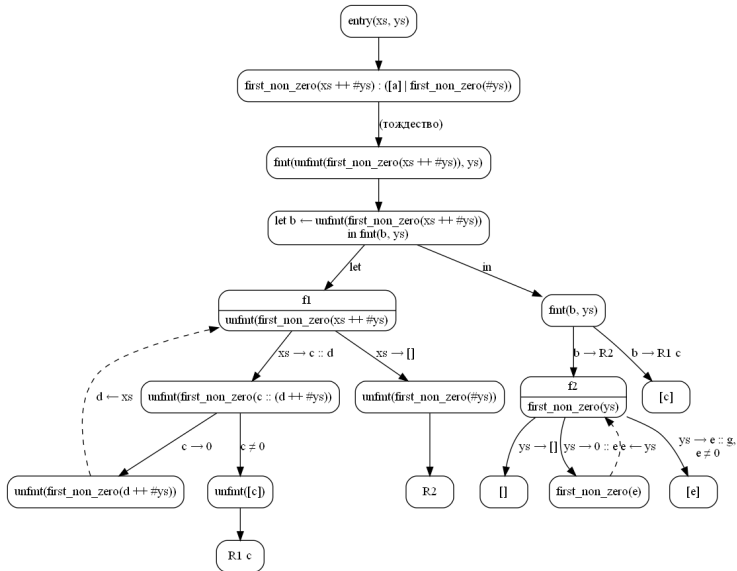
```
fun unfmt [a] = R1 a  
    | unfmt first_non_zero(#ys) = R2
```

```
fun fmt (R1 a, ys) = [a]  
    | fmt (R2, ys) = first_non_zero(#ys)
```

Продолжим суперкомпиляцию...

Пример на альтернативные форматы (8)

Окончательный граф суперкомпиляции имеет вид:



Пример на альтернативные форматы (9)

Остаточная программа будет иметь вид

```
datatype first_non_zero_ret = R1 int | R2
```

```
fun entry (xs, ys) =  
    case (f1 xs) of  
        (R1 c)  $\Rightarrow$  [c]  
    | R2  $\Rightarrow$  (f2 ys)
```

```
fun f1 (0 :: d) = (f1 d)  
    | f1 (c :: d) = (R1 c)  
    | f1 [] = R2
```

```
fun f2 [] = []  
    | f2 (0 :: e) = (f2 e)  
    | f2 (e :: g) = [e]
```

Пример на альтернативные форматы (10)

Функция `f2` тривиально совпадает с функцией `first_non_zero`, т.к. получена из соответствующего тривиального вызова, оптимизировать в ней нечего.

Функция `f1`, почти совпадающая с `f2`, появилась в процессе декомпозиции.

Функция `f2` не вызывается из входной точки, если в `xs` обнаружилось ненулевое число.

Альтернативные форматы (3)

Заметим, что альтернативные форматы расширяют домен типов данных суперкомпилируемой программы. Другие методы суперкомпиляции, известные докладчику, заменяют лишь домен функций, домен типов данных остаётся прежним (либо может сужаться, если суперкомпилятор вычищает типы, не используемые в остаточной программе).

Здесь же неизбежно добавляется новая директива `datatype`.

Можно заметить, что домен данных могут расширять и простые форматы, добавляя в остаточную программу типы кортежей, отсутствовавших в исходной программе, но кортежи — встроенный тип, поэтому в глаза не бросается.

Выводы

Выводы

- ▶ Предложен способ онлайн-вычисления форматов конфигураций в процессе суперкомпиляции.
- ▶ Форматы могут быть простыми, параметризованными, активными и альтернативными, каждый следующий класс включает предыдущий.
- ▶ Использование активных форматов позволяет выполнять декомпозицию функций в процессе суперкомпиляции.